Question 25

GH-300: Security & Code Quality

How does GitHub Copilot help prevent common security vulnerabilities (e.g., SQL Injection, Hardcoded Secrets, Insecure Deserialization) during code generation?

A. By executing runtime sandbox testing before suggesting code.

✔ **B. By utilizing a post-processing safety model that detects and blocks vulnerable code patterns (CWEs) in real time.**

C. By automatically attaching CodeQL security alert badges to source files.

D. By scanning for vulnerabilities only when code is pushed to a remote repository.

Correct Answer: B

GitHub Copilot integrates an inline post-processing engine designed to filter out insecure coding patterns. This safety model identifies common vulnerability types (such as SQL injections or hardcoded credentials) and prevents them from appearing in ghost text suggestions.

Option Breakdown

- **A. Runtime sandbox testing:** Incorrect. Copilot does not execute runtime binary tests for inline completions.
- **B. Real-time post-processing safety model:** Correct. Filters suggestions against Common Weakness Enumeration (CWE) security patterns.
- **C. Attaching CodeQL badges:** Incorrect. CodeQL runs in GitHub Actions workflows, not inside Copilot inline code output.
- **D. Scanning only on git push:** Incorrect. Safety filtering occurs in real time before code is shown in the IDE.

💡 **GH-300 Exam Tip**
Copilot's AI safety system filters out top security vulnerabilities, including **CWE-79 (XSS)**, **CWE-89 (SQL Injection)**, and **CWE-798 (Hardcoded Credentials)** before delivering recommendations.